

Análise Comparativa do HAProxy e do Nginx

Desempenho e Resiliência como Balanceadores de Carga sob Testes de Estresse e Injeção de Falhas

Universidade Federal de Campina Grande (UFCG) – CEEI / Centro de Informática

Disciplina: Sistemas Distribuídos

Arthur Vieira, Heitor Alves, Isaque Cavalcante, Rafael Alencar

Introdução e Contextualização

- **O Contexto Atual:** Crescimento acelerado de aplicações web exige arquiteturas de alta disponibilidade baseadas em balanceadores de carga.
- **HAProxy:** Proxy focado em alta disponibilidade, suporte avançado a *health checks* ativos e métricas detalhadas.
- **Nginx:** Servidor orientado a eventos (*event-driven*), reconhecido por sua alta eficiência com múltiplas conexões simultâneas.
- **Avaliação:** Utilização do *Grafana k6* para simulação real de tráfego sintético, medindo latência e throughput.



A Questão de Pesquisa

Como o HAProxy e o Nginx diferem em termos de desempenho, eficiência e resiliência na distribuição de tráfego quando submetidos a testes de carga e injeção de falhas? //

Superando escolhas baseadas no "senso comum" através de dados quantitativos diretos.

Objetivos do Estudo



Detecção de Falhas

Avaliar a velocidade de detecção e remoção de nós de backend que sofreram falha através de Health Checks.



Latência e Vazão

Medir o impacto no tempo de resposta (P95 e P99) e vazão sob picos repentinos de acesso (Flash Crowd).



Recuperação

Analisar a capacidade de recuperação e tolerância a falhas de rede versus falhas de processo.

Arquitetura Experimental

1. Gerador de Carga (Grafana k6)

Responsável por simular os *Virtual Users (VUs)* e registrar os dados brutos de desempenho e latência em formato JSON.

2. Camada de Balanceamento

Instância isolada (via Docker Compose) operando o HAProxy ou o Nginx para orquestrar a distribuição do tráfego.

3. Camada de Aplicação

Conjunto de réplicas de microsserviços rodando em containers como *backends* da arquitetura.

4. Chaos Engineering (Pumba)

Ferramenta utilizada para injetar falhas e degradações controladas diretamente nos containers de backend.

Cenários de Teste



Baseline (Linha de Base)

Carga nominal e estável utilizada como grupo de controle para estabelecer os limites normais do sistema sem anomalias.



Spike (Pico de Carga)

Elevação abrupta de VUs (rampas de 5→50 e 100→400 VUs) para testar o esgotamento da fila de conexões.



Failure (Injeção via Pumba)

Falhas de Rede (delay e loss) e Falhas de Disponibilidade/Processo (kill, stop, pause) nos nós de backend.

Resultados: Cenário Baseline (Operação Normal)

~50

Requisições / Segundo

Empate Técnico

Em condições normais de operação, as duas tecnologias mostraram-se perfeitamente intercambiáveis, operando com **0% de erro**.

Latência Média: HAProxy (29,29 ms) vs Nginx (29,41 ms)

Latência de Cauda (P99): HAProxy (51,39 ms) vs Nginx (51,78 ms)

Resultados: Cenário Spike (Pico de Tráfego)

HAProxy: Absorção por Fila

O HAProxy prioriza manter as conexões abertas. Ele absorve a sobrecarga retendo os pacotes, resultando em uma **baixa taxa de erro geral (2,53%)**. No entanto, o custo dessa abordagem é uma latência de cauda significativamente elevada, com o percentil P99 chegando próximo a 4.742 ms.

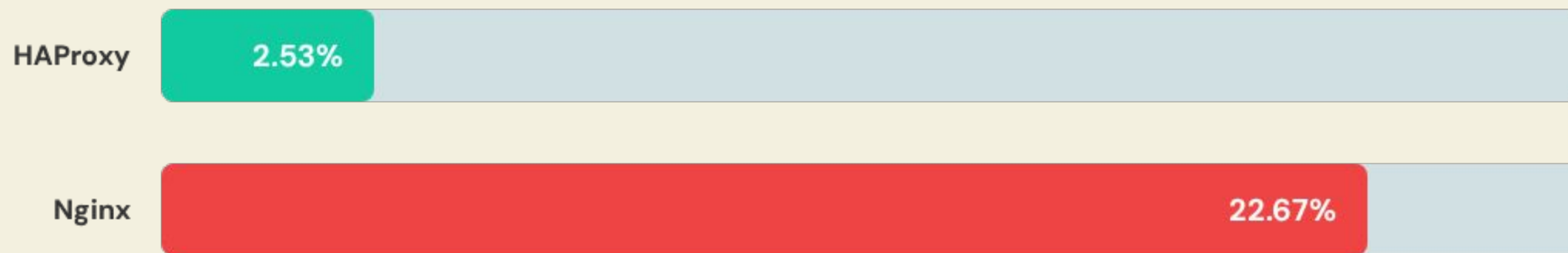
Nginx: Fail-Fast (Rejeição Rápida)

O Nginx rejeita rapidamente as requisições quando esgota sua capacidade (backlog). Isso gera uma **alta taxa de erro geral (22,67%)**, chegando a 45,55% no teste mais rigoroso. O benefício dessa rejeição é manter a latência sob controle para os pacotes processados (P99 de 679 ms).

Resultados: Falhas de Rede e Disponibilidade

-  **Falhas de Rede (Delay e Loss):** Ambos os balanceadores apresentaram 0% de erro. A perda de pacotes foi amortecida nativamente pelas retransmissões TCP, e o atraso (delay) refletiu apenas em uma maior latência, sem derrubar a aplicação.
-  **Falhas de Disponibilidade (Kill, Stop, Pause):** Na configuração padrão, o Nginx lidou de forma mais veloz, mantendo 0% de erros perante as quedas de backend.
-  **Detecção no HAProxy:** O HAProxy levou mais tempo para remover o nó do cluster após a falha do processo. Isso gerou pequenas taxas de erro (0,41% a 0,52%) e janelas ligeiramente maiores de degradação até que o tráfego fosse totalmente redirecionado.

Síntese Comparativa: Taxa de Erro (Spike)



A discrepância na taxa de erro ilustra perfeitamente as filosofias arquiteturais distintas: o HAProxy prefere atrasar a resposta em vez de negá-la, enquanto o Nginx interrompe o processamento para não gargalar o servidor.

Lições Aprendidas e Conclusões

Não existe "vencedor absoluto": A escolha tecnológica depende estritamente das regras de negócio e da tolerância a falhas da aplicação.

Aplicações Críticas (Ex: Financeiras): Beneficiam-se do HAProxy, pois toleram atraso no processamento, mas evitam falhas explícitas ao usuário final.

Alta Responsividade (Ex: Streaming/Games): Beneficiam-se do Nginx, falhando rápido e permitindo que o cliente gerencie o erro (retries) sem travamentos.



Limitações e Trabalhos Futuros

Limitações Observadas

- Execução em ambiente virtualizado com medições de execução única (falta de intervalos de confiança).
- Efeitos de borda causados pelo comportamento do k6 durante retours e timeouts das requisições sintéticas.

Trabalhos Futuros

- Realizar testes modificando parâmetros de Active vs. Passive Health Checks em ambos os proxies.
- Avaliação profunda do impacto no uso de CPU e Memória por nó durante as janelas de degradação.

Obrigado!

Principais referências:

AJAZ, Ubaid et al. *Optimizing Load Balancing Framework*, 2026.

PRAMONO, L. et al. *Round-robin Algorithm in HAProxy and Nginx*, 2018.

ZULMAN, M. R. et al. *Performance and Fault Tolerance Analysis*, 2026.